AVCRI Project — Complete Database Audit & Schema Guide

This document provides a complete, presentation-ready breakdown of the **AVCRI** database architecture, every table and column group, all foreign keys (`FK`), many-to-many (`M2M`) relationships, unique constraints, cross-database logical links, live row counts from `db.sqlite3` , and key schema audit findings.

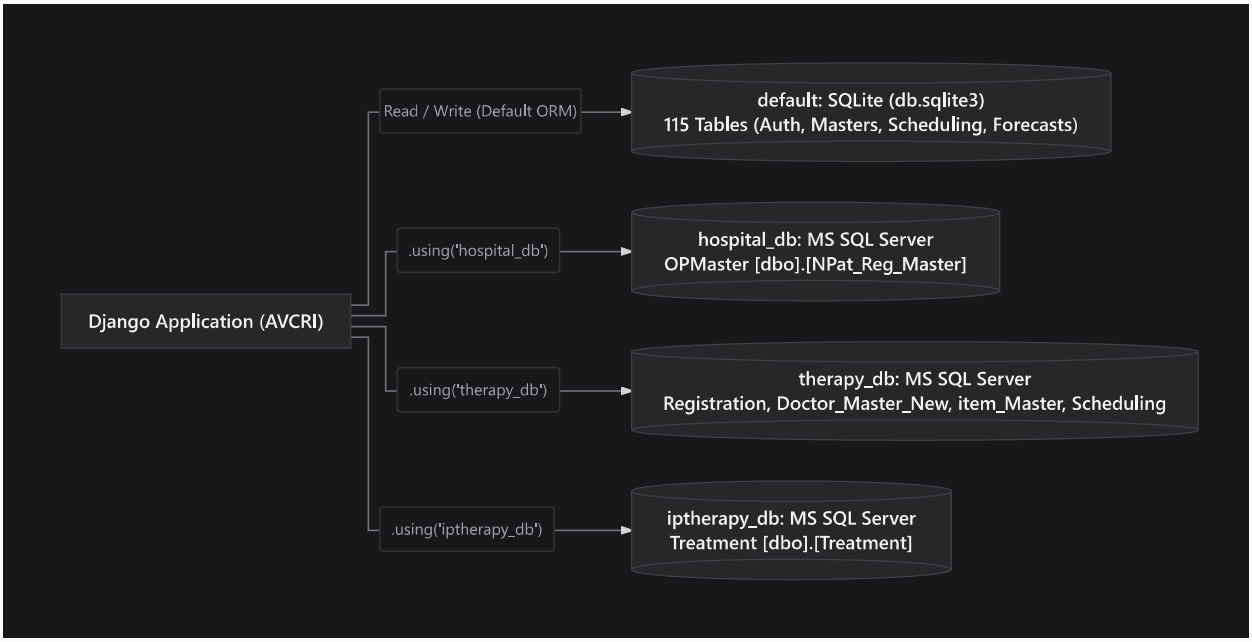
1. High-Level Multi-Database Architecture

Configured in , the project uses **4 database connections** with **no** `DATABASE_ROUTERS` — code explicitly chooses external databases using `.using('<alias>')` or raw `connections['<alias>']` :

Alias	Engine	Purpose	Managed by Django?
`default`	SQLite (`db.sqlite3`)	All application operational data, users/RBAC, local masters, scheduling, bookings, prescriptions, forecasts, and audit logs (**115 tables total**).	**Yes** (`managed = True` , migrations active; `makemigrations --check` reports in sync)
`hospital_db`	MS SQL Server (`mssql` / ODBC 18)	**OPTherapy** external hospital database — used for Outpatient (`OPMaster -> [dbo].[NPat_Reg_Master]`).	**No** (`managed = False` , read/lookup)
`therapy_db`	MS SQL Server (`mssql` / ODBC 18)	**Therapy-new** external hospital database — used for Inpatient registration (`Registration`), doctors (`doctor_master`), external item/scheduling lookups (`ItemMaster` , `Scheduling`).	**No** (`managed = False` , read/lookup)
`iptherapy_db`	MS SQL Server (`mssql` / ODBC 18)	**Therapy-new** external SQL Server connection (`convert_tz: False`) — used specifically for external `Treatment` records (`[dbo].[Treatment]`).	**No** (`managed = False` , read/lookup)

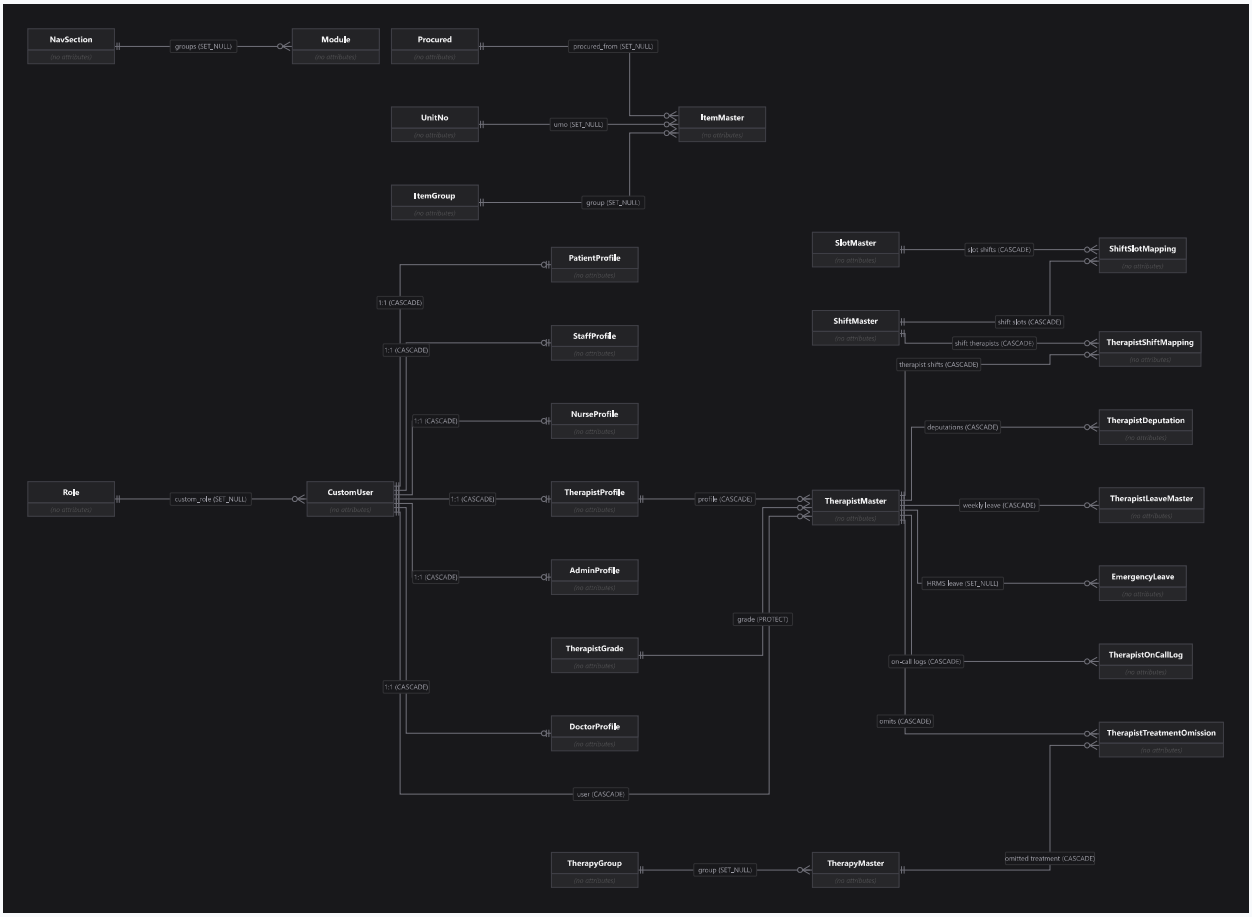
NOTE

> Even though one Django app is named `MySql_Data_pull`, all three external database connections in `settings.py` use Microsoft SQL Server (`mssql` with `ODBC Driver 18 for SQL Server`), not MySQL.

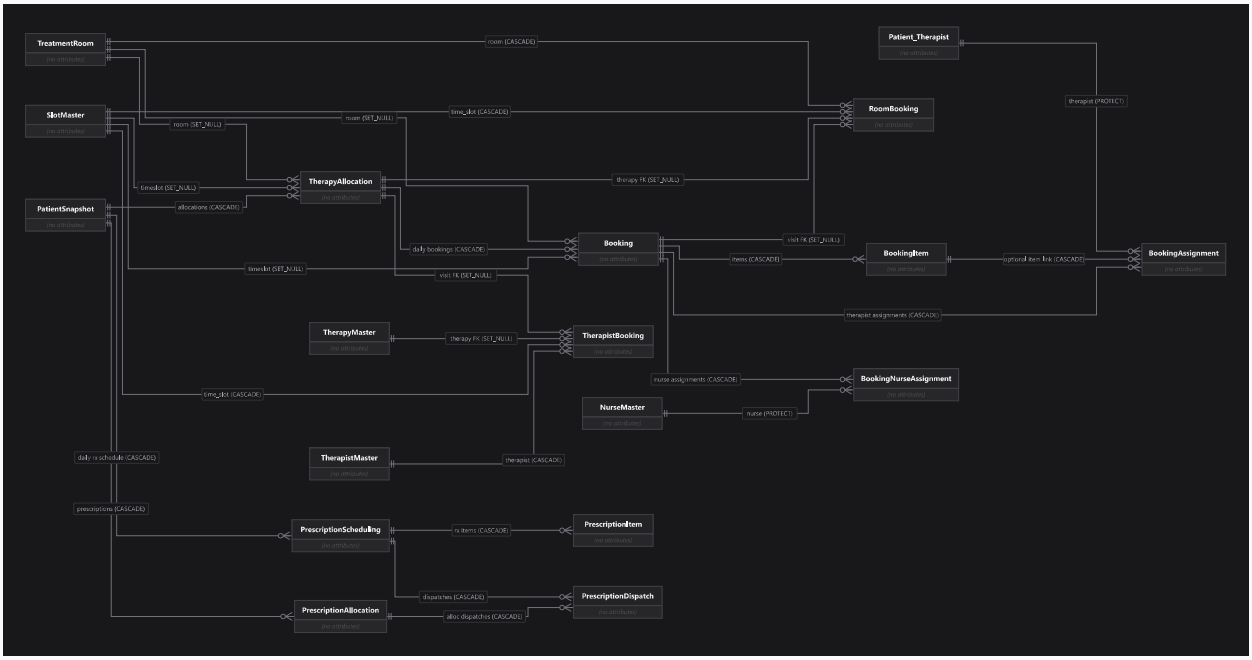


2. Core Entity-Relationship Overview

2.1 Authentication, RBAC & Master Data Relationships



2.2 Therapy Scheduling, Bookings & Prescriptions



3. Complete App-by-App Table & Foreign Key Breakdown

All row counts below were observed directly from `db.sqlite3` during this audit.

App 1: `authentication` ()

Handles custom user accounts, Role-Based Access Control (RBAC), dynamic sidebar navigation, user role profiles, and system audit logs.

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys (`FK` / `1:1` / `M2M`)
	`authentication_navsection`	7	`id`	`name` (unique), `display_name`, `icon`, `order`, `is_active`	None
	`authentication_module`	48	`id`	`name` (unique), `display_name`, `url_name`, `url_path`, `icon`, `order`, `is_active`	`nav_section` -> `NavSection` (`SET_NULL`, nullable)
	`authentication_role`	27	`id`	`name` (unique), `display_name`, `description`, `is_active`, timestamps	`modules` <-> `Module` (**M2M** via `authentication_role_modules`, 163 rows)
	`authentication_customuser`	238	`id`	Extends `AbstractUser`: `username`, `role` (13 choices), `mobile`, `profile_picture`, `gender`	`custom_role` -> `Role` (`SET_NULL`, nullable); plus Django `groups` (`authentication_customuser_groups`) & `user_permissions` (`authentication_customuser_user_permissions`)
	`authentication_auditlog`	21,350	`id`	`action`, `description`, `ip_address`, `user_agent`, `status`, `timestamp`, `object_type`, `object_id`, `additional_data` (JSON)	`user` -> `CustomUser` (`SET_NULL`, nullable)
	`authentication_patientprofile`	13	`id`	`patient_reg_no` (unique), `date_of_birth`, `gender`, `address`, `emergency_contact`, `emergency_phone`, `medical_history`, `allergies`, `is_vip`	`user` -> `CustomUser` (**1:1**, `CASCADE`)
	`authentication_therapistprofile`	138	`id`	`therapist_code` (unique), `employee_code` (unique, nullable), `therapist_grade`, `department`, `specialization`, `is_bathing_therapist`, `joining_date`, `qualification`, `experience_years`	`user` -> `CustomUser` (**1:1**, `CASCADE`)
	`authentication_doctorprofile`	54	`id`	`doctor_id` (unique), `specialization`, `department`, `qualification`, `experience_years`, `license number`, `consultation_fee`	`user` -> `CustomUser` (**1:1**, `CASCADE`)

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys (`FK` / `1:1` / `M2M`)
	`authentication adminprofile`	8	`id`	`admin_id` (unique), `department`, `designation`, `reporting_manager`	`user` -> `CustomUser` (**1:1**, `CASCADE`)
	`authentication nurseprofile`	1	`id`	`nurse_id` (unique), `department`, `qualification`, `experience_years`, `shift`	`user` -> `CustomUser` (**1:1**, `CASCADE`)
	`authentication staffprofile`	13	`id`	`staff_id` (unique), `department`, `designation`, `reporting_manager`	`user` -> `CustomUser` (**1:1**, `CASCADE`)

App 2: `masters` ()

Contains the **primary operational master tables** used across therapy scheduling, pharmacy/procurement, HR leave/deputation, and shift rules.

2A. Lookup & Pharmacy/Prescription Masters

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	`masters_procured`	2	`id`	`name` (unique, e.g., IP Pharmacy, Kashaya Room), `is_active`	None
	`masters_oilclass`	12	`id`	`name` (unique), `is_active`	None
	`masters_usagetype`	52	`id`	`name` (unique), `is_active`	None
	`masters_prescriptionusage`	58	`id`	`name` (unique), `order_no`, `is_active`	None
	`masters_prescriptiontime`	5	`id`	`name` (unique, e.g., 8 AM, 12 PM), `order_no`, `is_active`	None
	`masters_prescriptionscope`	2	`id`	`name` (unique, Ext/Int), `is_active`	None
	`masters_unitno`	20	`id`	`unit_name` (unique, e.g., ml, cm), `is_active`	None
	`masters_itemgroup`	5	`id`	`name` (unique), `is_active`	None
	`masters_itemmaster`	1,371	`id`	`item_code` (unique 4-digit auto-generated), `item_name`, `min_quantity`, `max_quantity`, `is_active`	`item_group` -> `ItemGroup` (`SET_NULL`); `umo` -> `UnitNo` (`SET_NULL`); `procured_from` -> `Procured` (`SET_NULL`)
	`masters_ingredientmaster`	3	`id`	`ingredient_code` (unique), `ingredient_name`, `description`, `is_active`	None
	`masters_stayingroom`	99	`id`	`number` (unique), `room_desc`, `room_no`, `is_active`	None

2B. Therapy, Slot, Shift & Nurse Masters

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	`masters_therapygroup`	9	`id`	`name` (unique), `is_active`	None
	`masters_therapymaster`	238	`id`	`therapy_code` (unique), `therapy_name`, `therapy_category` (Easy/Medium/Hard), `therapy_duration`, `bathing_required`, `treatment_room_required`, `cost_of_therapy`, `patient_type` (`IP` / `OP`), therapist grade counts (`no_of_super_senior/senior/junior/trainee/total`), `is_steam_therapy`, `is_vamanam`, `is_vasthi`, `is_active`	`therapy_group -> TherapyGroup` (`SET_NULL`, nullable)
	`masters_nursetreatmentmaster`	15	`id`	`treatment_code` (unique), `name`, `number_of_nurse_required`, `is_active`	None
	`masters_nursemaster`	6	`id`	`nurse_code` (unique), `nurse_name`, `designation`, `is_active`	None
	`masters_slotmaster`	9	`id`	`slot_number` (unique), `slot_time`, `slot_name`, `is_vamanam_slot`, `is_vasthi_slot`, `is_active`	None
	`masters_vamanamtimingmaster`	2	`id`	`timing name`, `timing_time`, `display_order`, `is_active`	None
	`masters_shiftmaster`	2	`id`	`shift_name` (unique), `is_active`	None
	`masters_shiftslotmapping`	14	`id`	`is_active`; Unique `(shift, slot)` (`ux_shift_slot_unique`)	`shift -> ShiftMaster` (`CASCADE`); `slot -> SlotMaster` (`CASCADE`)

2C. Therapist Master, Shifts, Leave, Deputation & Omissions

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	masters_therapistgrade	4	id	name (unique), display_order, is_active	None
	masters_therapistmaster	124	id	therapist_code (unique), therapist_name, therapist_gender, employee_code (unique, nullable), is_bathing_therapist, is_helping_therapist, is_deputed, is_on_call, on_call_status, on_call_start_date, on_call_end_date, is_active	user -> CustomUser (CASCADE, nullable); therapist_profile -> TherapistProfile (CASCADE, nullable); therapist_grade -> TherapistGrade (PROTECT, nullable)
	masters_therapistshiftmapping	154	id	is_active; Unique (therapist, shift) (ux_therapist_shift_unique)	therapist -> TherapistMaster (CASCADE); shift -> ShiftMaster (CASCADE)
	masters_therapistleavemaster	117	id	leave_day (0–6 Mon–Sun), leave_day_name, is_active; Unique (therapist, leave_day)	therapist -> TherapistMaster (CASCADE)
	masters_emergencyleave	2,493	id	leave_id (unique), employee_code (indexed), from_date, to_date, is_half_day, half_day_date, half_day_session (Forenoon / Afternoon / Both), custom_forenoon, custom_afternoon, leave_type, status, raw_payload (JSON)	therapist -> TherapistMaster (SET_NULL, nullable)
	masters_therapistoncalllog	6	id	status (turned_on / turned_off), start_date, end_date, remarks, created_at	therapist -> TherapistMaster (CASCADE); changed_by -> CustomUser (SET_NULL)
	masters_therapistdeputation	0	id	destination, start_date, duration_days, end_date, assigned_doctor_name, remarks, status (pending / approved / rejected), approved_at, approval_remarks	therapist -> TherapistMaster (CASCADE); assigned_doctor -> CustomUser (SET_NULL); requested_by -> CustomUser (SET_NULL); approved_by -> CustomUser (SET_NULL)

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	`masters_therapisttreatmentomission`	218	`id`	`reason`, `status` (`pending` / `approved` / `rejected`), `approved_at`, `approval_remarks`, `batch_id` (indexed), `is_active`	`therapist` -> `TherapistMaster` (`CASCADE`); `treatment` -> `TherapyMaster` (`CASCADE`); `requested_by` -> `CustomUser` (`SET_NULL`); `approved_by` -> `CustomUser` (`SET_NULL`)

App 3: `Patient` ()

Contains **Treatment Room masters**, **Booking resource locks (`RoomBooking`, `TherapistBooking`)**, historical patient data, cancellation audits, allocation attempt logs, and several **legacy master tables** being phased out in favor of `masters`.

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	`Patient roomname`	0	`id`	`name` (unique), `is_active`	None
	`Patient_floor`	5	`id`	`name` (unique), `is_active`	None
	`Patient treatmentroom`	43	`room_no` (Char 20)	`floor` (legacy string), `description`, `tr_code` (auto 4-digit), `is_steam_room`, `is vamanam room`, `vamanam_room_type` (`primary` / `secondary`), `is_wheelchair_accessible`, `is_active`	`room_name` -> `RoomName` (`SET_NULL`); `floor_master` -> `Floor` (`SET_NULL`)
	`Patient roombooking`	7,272	`id`	`date`, `therapy_name`, `patient_id` (BigInt, indexed), `status` (`BOOKED` / `CANCELED` / `COMPLETED`). Unique `(room, date, time_slot, status)`	`room` -> `TreatmentRoom` (`CASCADE`); `time slot` -> `masters.SlotMaster` (`CASCADE`); `visit` -> `Sheduling.Booking` (`SET_NULL`, nullable); `therapy` -> `Sheduling.TherapyAllocation` (`SET_NULL`, nullable)
	`Patient therapistbooking`	21,138	`id`	`date`, `therapy_name`, `patient_id` (BigInt, indexed), `status`, `is bathing`, `is_helping_therapist_assignment`. Unique `(therapist, date, time_slot, status)`	`therapist` -> `masters.TherapistMaster` (`CASCADE`); `time slot` -> `masters.SlotMaster` (`CASCADE`); `visit` -> `Sheduling.TherapyAllocation` (`SET_NULL`, nullable); `therapy` -> `masters.TherapyMaster` (`SET_NULL`, nullable)
	`Patient repeated visitors`	3,625	`register number` (Int)	`patient_name`, `total_visits`, `is_vip`	None
	`Patient patienthistory`	262,000	`id`	`preg_no` (indexed), `pat_name`, `date`, `therapist_name`, `therapist_code`, `room no`, `treatment`, `t_room`	None (historical flat table; largest table in DB)
	`Patient_dailyp`	1,096	`id`	`date` (unique), `patient_count`,	None

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	atientcount			created_at	
	Patientbookingcancellations	76	id	canceled_at, reason, patient_id (indexed), date, time_slot_id (Int), therapy_name, therapist_name, room_no	canceled_by -> CustomUser (SET_NULL, nullable)
	Patientallocationattempt	295	id	date, requested_time_slot_value, status (SUCCESS / FAILED), reason, extra (JSON)	visit -> MySql_Data_pull.Visit (SET_NULL); therapy -> MySql_Data_pull.store_Therapy (SET_NULL); requested time slot -> Patient.TimeSlot (SET_NULL)
(Legacy)	Patienttherapist	120	therapist_code (Char 20)	therapist_name, therapist grade, is_bathing_therapist	None *(still referenced by Sheduling.BookingAssignment & Patient.TherapistLeave)*
(Legacy)	Patienttherapistleave	105	id	leave day (0–6), leave_day_name	therapist -> Patient.Therapist (CASCADE)
(Legacy)	Patienttherapydetails	140	therapy_code (Char 20)	therapy_name, therapy_group, therapy_category, therapy_duration, bathing required, treatment_room_required, cost_of_therapy, therapist counts	None *(superseded by masters.TherapyMaster)*
(Legacy)	Patienttimeslot	7	slot_number (Int)	start_time, end_time	None *(superseded by masters.SlotMaster)*

App 4: Sheduling ()

(Note: spelled Sheduling in code/DB). This is the **core operational engine** for therapy allocations, daily bookings, therapist/nurse assignments, continuity planning, preferred therapist rosters, prescription scheduling/dispatch, and scheduling audit trails.

4A. Patient Snapshot & Therapy Scheduling Core

Model	SQLite Table	Rows	PK	Key Columns & Constraints	Foreign Keys
	`Scheduling patientsnapshot`	1,813	`id`	`pat_reg_no` (unique String), `pat_name`, `pat_age`, `pat_sex`, `pat_room_no`, `pat ref id`, `is wheelchair`, `is_vip`, `source_updated_at`	None *(local cache of external patient info)*
	`Scheduling therapyallocation`	3,812	`id`	`therapy_code`, `therapy_name`, `ext_int`, `items_groups` (JSON), `oil_class`, `usage`, `procured_from`, `from_date`, `to_date`, `days`, `required_therapist_roles` (JSON), `locked`, `vamanam_timing`, `is_wheelchair`, `is_vip`, `bathing_therapists_count`, `status` (`draft` / `allocated` / `cancelled` / `failed`), cancellation/resume metadata	`patient` -> `PatientSnapshot` (`CASCADE`); `timeslot` -> `masters.SlotMaster` (`SET NULL`); `room` -> `Patient.TreatmentRoom` (`SET_NULL`); `allocated_by`, `cancelled_by`, `resumed by` -> `CustomUser` (`SET_NULL`)
	`Scheduling booking`	22,196	`id`	`date` (indexed), `vamanam_timing`, `is_wheelchair`, `is_vip`, `bathing_therapists_count`, `status`, `therapist_note`, `failure_reason`, `snapshot therapy name`. **Unique `(room, date, timeslot)`** (`ux_booking_room_date_timeslot`)	`allocation` -> `TherapyAllocation` (`CASCADE`); `timeslot` -> `masters.SlotMaster` (`SET NULL`); `room` -> `Patient.TreatmentRoom` (`SET_NULL`)
	`Scheduling bookingitem`	14,442	`id`	`item code`, `item_name`, `dosage`, `uno`, `status` (`scheduled` / `cancelled` / `completed`), `cancel_reason`, timestamps	`booking` -> `Booking` (`CASCADE`); `cancel_requested_by`, `cancelled_by`, `completed by` -> `CustomUser` (`SET_NULL`)
	`Scheduling bookingassignment`	20,105	`id`	`role`, `assigned_for` (`booking` / `booking_item`), `status` (`assigned` / `released`), `booking_date` (denormalized). **Unique `(therapist, booking_date, booking_timeslot)`** (`ux_assignment_therapist_date_timeslot`)	`booking` -> `Booking` (`CASCADE`); `booking item` -> `BookingItem` (`CASCADE`, nullable); **`therapist` -> `Patient.Therapist` (`PROTECT`)**; `released by` -> `CustomUser` (`SET_NULL`); `booking timeslot` -> `masters.SlotMaster` (`SET_NULL`)

Model	SQLite Table	Rows	PK	Key Columns & Constraints	Foreign Keys
	`Scheduling bookingnurseassignment`	634	`id`	`status` (`assigned` / `released`), `booking_date`. **`Unique (nurse, booking)`** (`ux_nurse_assignment_nurse_booking`)	`booking` -> `Booking` (`CASCADE`); `nurse` -> `masters.NurseMaster` (`PROTECT`); `released by` -> `CustomUser` (`SET_NULL`); `booking timeslot` -> `masters.SlotMaster` (`SET_NULL`)
	`Scheduling therapistcontinuityplan`	19,174	`id`	`date`, `therapy_name`, `is_bathing`, `priority` (0/1/2), `source_date`	`time slot` -> `masters.SlotMaster` (`CASCADE`); `patient` -> `PatientSnapshot` (`CASCADE`); `allocation` -> `TherapyAllocation` (`CASCADE`); `therapist` -> `masters.TherapistMaster` (`CASCADE`)

4B. Cancellations, Reschedules, Audits & Final Run Markers

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	`Scheduling cancellationrequest`	0	`id`	`from_date`, `to_date`, `items` (JSON), `reason`, `status` (`pending` / `approved` / `rejected`), `admin_action_at`	`allocation` -> `TherapyAllocation` (`CASCADE`); `requested_by`, `admin action_by` -> `CustomUser` (`SET_NULL`)
	`Scheduling bookingrescheduleh istory`	11	`id`	`before_date`, `before_therapists` (JSON), `after_date`, `after_therapists` (JSON), `rescheduled_at`, `reason`	`booking` -> `Booking` (`CASCADE`); `allocation` -> `TherapyAllocation` (`CASCADE`); `before_slot`, `after slot` -> `masters.SlotMaster` (`SET_NULL`); `before_room`, `after room` -> `Patient.TreatmentRoom` (`SET_NULL`); `rescheduled_by` -> `CustomUser` (`SET_NULL`)
	`Scheduling allocationchange`	1	`id`	`change type`, `diff` (JSON), `reason`, `created_at`	`allocation` -> `TherapyAllocation` (`CASCADE`); `changed_by` -> `CustomUser` (`SET_NULL`)
	`Scheduling auditlog`	17,631	`id`	`action`, `obj_type`, `obj_id`, `data` (JSON), `created_at`	`user` -> `CustomUser` (`SET_NULL`)
	`Scheduling schedulingauditlog`	11	`id`	`change_type` (`emergency_replacement` / `bathing_reassignment` / `reschedule` / `allocation_change`), patient/therapy/schedule snapshots, replaced vs replacement therapist snapshots, `before_state` & `after_state` (JSON), `reason`	`booking` -> `Booking` (`SET_NULL`); `allocation` -> `TherapyAllocation` (`SET_NULL`); `changed by` -> `CustomUser` (`SET_NULL`)
	`Scheduling helpingtherapistallocationlog`	1	`id`	`patient_id`, `treatment_id`, `visit_id`, `booking_id`, `therapy_name`, `date`, `time_slot`, `reason_for_fallback`, `shortage condition`, `is_supporting_backup`	`helping therapist` -> `masters.TherapistMaster` (`CASCADE`)

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	`Scheduling finalscheduleRun`	81	`id`	`run_date` (unique), `status` (`success` / `failed`), `ran_at`, `message`, `stats` (JSON)	`ran_by` -> `CustomUser` (`SET_NULL`)
	`Scheduling prescriptionfinalschedulerun`	68	`id`	`run_date` (unique), `status` (`success` / `failed`), `ran_at`, `message`, `stats` (JSON)	`ran_by` -> `CustomUser` (`SET_NULL`)

4C. Therapist Preferences & Doctor Quotas

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys & M2M
	`Scheduling doctorpatientpreferenceallowance`	3	`id`	`doctor_code`, `doctor_name`, `patient_reg_no`, `patient_name`, `year`, `month`, `is_active` (5/month quota). Unique `(doctor, patient_reg_no, year, month)`	`doctor` -> `CustomUser` (`CASCADE`)
	`Scheduling patientpreferredtherapistroster`	1	`id`	`patient_reg_no` (unique), `patient_name`, `therapist codes` (JSON), `unpreferred_therapist_codes` (JSON), `primary_bathing_code`, `secondary_bathing_code`, `is_active`	`allowance` -> `DoctorPatientPreferenceAllowance` (`SET NULL`); `primary_bathing_therapist` & `secondary_bathing_therapist` -> `masters.TherapistMaster` (`SET_NULL`); `configured_by` -> `CustomUser` (`SET_NULL`); **M2M** `therapists` (7 rows) & `unpreferred_therapists` (0 rows) -> `masters.TherapistMaster`
	`Scheduling therapistpreferencerequest`	0	`id`	`patient_id`, `patient_name`, `treating_doctor_code/name`, `treatment_name`, code/name JSON snapshots, `status`, `is_active`, approval/deactivation fields	`visit` -> `MySql_Data_pull.Visit` (`SET_NULL`); `requested_by` & `approved_by` -> `CustomUser` (`SET_NULL`); **M2M** `preferred_therapists` & `unpreferred therapists` -> `masters.TherapistMaster`
	`Scheduling therapistpreferenceauditlog`	0	`id`	`patient_id`, `action`, `action_by_name`, `action_date`, `details` (JSON), `notes`	`preference request` -> `TherapistPreferenceRequest` (`CASCADE`); `action_by` -> `CustomUser` (`SET_NULL`)

4D. Prescriptions & Notifications

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys & M2M
	`Scheduling prescriptionallocation`	5,314	`id`	`prescription_name`, `items_groups` (JSON), `procured_from`, `usage`, `scope`, `from_date`, `to_date`, `days`, `locked`, `status` (`draft` / `active` / `cancelled` / `failed`), cancel/resume fields	`patient` -> `PatientSnapshot` (`CASCADE`); `allocated_by`, `cancelled_by`, `resumed_by` -> `CustomUser` (`SET_NULL`)
	`Scheduling prescriptionscheduling`	18,667	`id`	`prescription_date`, `status` (`draft` / `scheduled` / `dispatched`), `sent_to`, `dispatched_from`, `dispatched_at`. Unique `(patient, prescription_date)`	`patient` -> `PatientSnapshot` (`CASCADE`); `doctor` -> `CustomUser` (`CASCADE`); `scheduled_by` -> `CustomUser` (`SET_NULL`)
	`Scheduling prescriptionitem`	0	`id`	`item_name`, `dose`, `quantity`, `time`, `notes`	`prescription_scheduling` -> `PrescriptionScheduling` (`CASCADE`)
	`Scheduling prescriptiondispatch`	11	`id`	`department`, `dispatched_at`, `notes`	`prescription_scheduling` -> `PrescriptionScheduling` (`CASCADE`); `prescription_allocation` -> `PrescriptionAllocation` (`CASCADE`, nullable); `dispatched_by` -> `CustomUser` (`CASCADE`)
	`Scheduling prescriptionchangelog`	19,363	`id`	`prescription_date`, `action` (`create` / `update` / `delete` / `status_change`), `before_values` & `after_values` (JSON), `timestamp`, `description`	`patient` -> `PatientSnapshot` (`CASCADE`); `changed_by` -> `CustomUser` (`CASCADE`)
	`Scheduling prescriptionchange`	0	`id`	`change_type`, `diff` (JSON), `reason`, `created_at`	`prescription_allocation` -> `PrescriptionAllocation` (`CASCADE`); `changed_by` -> `CustomUser` (`SET_NULL`)
	`Scheduling notificationqueue`	23	`id`	`target_type`, `target_identifier`, `channel`, `payload` (JSON), `status`, `attempts`, `last_attempt_at`	None

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys & M2M
	`Scheduling therapynotification`	0	`id`	`notification_type`, `title`, `message`, `priority`, `target_roles` (JSON), `is_read`, `is_sent`, `sent_at`	`therapy_allocation` -> `TherapyAllocation` (`CASCADE`); `created_by` -> `CustomUser` (`SET_NULL`); **M2M** `target_users` -> `CustomUser`
	`Scheduling prescriptionnotification`	0	`id`	`notification_type`, `title`, `message`, `priority`, `target_roles` (JSON), `is_read`, `is_sent`, `sent_at`	`prescription_allocation` -> `PrescriptionAllocation` (`CASCADE`); `created_by` -> `CustomUser` (`SET_NULL`); **M2M** `target_users` -> `CustomUser`

App 5: `predictor` ()

Stores Prophet-based patient demand forecasting results and run logs.

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	`predictor_prediction`	1,324	`id`	`date`, `prediction_type` (`daily` / `weekly` / `monthly`), `predicted_value`, `yhat_lower`, `yhat_upper`, `actual_value`, `created at`. Unique `(date, prediction_type)`	None
	`predictor_forecastrun`	15	`id`	`run_type`, `status` (`pending` / `running` / `completed` / `failed`), `started_at`, `completed_at`, `error_message`, `start_date`, `end_date`, `periods`, `aggregate_by`, `predictions_created`, `predictions_updated`, `outliers_skipped`	`started_by` -> `CustomUser` (`SET_NULL`, nullable)

App 6: `MySql_Data_pull` ()

This app contains **two distinct groups** of models:

6A. External MS SQL Server Models (`managed = False` — Not in `db.sqlite3`)

Model	SQL Server Table	Target DB Alias	PK	Purpose
	`[dbo].[NPat Reg_Master]`	`hospital_db`	`NPat_Reg_No`	Outpatient registration master (name, sex, address, city, reg date).
	`[dbo].[Registration]`	`therapy_db`	`pat_reg_no`	Inpatient registration master (`pat_name`, `doctcode`, `pat_status='Y'` for active IP, `pat_room_no`, `pat_visit_date`, contact info).
	`[dbo].[Doctor Master_New]`	`therapy_db`	`Doct_Code`	Hospital doctor codes, names, and active status.
	`[dbo].[item_Master]`	`therapy_db`	`item_code`	External hospital item catalog (name, UOM, group, category, min/max/ROL, procured_from).
	`[dbo].[Scheduling]`	`therapy_db`	`pat_reg_no` (`PReg_No`)	Historical/external scheduling table used by `predictor/forecast.py` to pull patient counts.
	`[dbo].[Treatment]`	`iptherapy_db`	`Pat_Reg_No`	Inpatient treatment/admission records (`DOA`, `Treat_Doc_Code`, `Pat_Room_No`, `Pat_DoD`, `Treat_Item_Code`, etc.).
	`[dbo].[Deput_Details]`	External	`thpst_code`	External therapist deputation details.
	`[dbo].[Prescription]`	External	`pat_reg_no` (`Pat_Reg_No`)	External legacy prescription records.
	`[dbo].[Room_Master]`	External	`room_code`	External room master.
	`[dbo].[Therapist_Details]`	External	`therapist_code`	External therapist demographics, grade, weekly off, and phone.
	`[dbo].[Therapist_Master]`	External	`therapist class_code`	External therapist class master.
	`[dbo].[TR_Master]`	External	`TRCode`	External treatment room code master.
	`[dbo].[Slot_Maste`	External	`slot_no`	External time slot master *(flagged as low/unused)*.

Model	SQL Server Table	Target DB Alias	PK	Purpose
	r]			

6B. Local SQLite Visit/Store Models (`managed = True` — Stored in `db.sqlite3`)

Model	SQLite Table	Rows	PK	Key Columns	Foreign Keys
	MySql Data_pull_visit	20	id	patient_id (BigInt), patient_name, from_date, to_date, admission_days, is_ip_or_op, is_vip, repeated status, date_of_visit	None
	MySql Data pull store_therapy	31	id	patient_id, therapy_name, ext_int, procured, from date, to date, days, preferred_time_slot_value	visit -> Visit (CASCADE); preferred_time_slot -> Patient.TimeSlot (SET_NULL)
	MySql Data pull store_therapyitem	42	id	item_code, oil_class, usage, procured, from_date, to date, days, status, cancelled_at	therapy -> store_Therapy (CASCADE); cancelled_by -> CustomUser (SET_NULL)
	MySql Data pull store therapyitemday	26	id	date, status (active / cancelled / completed), cancelled at. Unique (therapy_item, date)	therapy item -> store_TherapyItem (CASCADE); cancelled_by -> CustomUser (SET_NULL)
	MySql Data pull store prescription	20	id	patient id (BigInt), date_created	visit -> Visit (CASCADE, nullable)
	MySql Data pull store prescriptionitem	22	id	item_code, dosage, usage, internal_external, time, item_uom, from_date, to_date, days, procured_from, status	prescription -> store_Prescription (CASCADE); cancelled_by -> CustomUser (SET_NULL)

Django Built-in & Orphan/Legacy SQLite Tables

In addition to the app models above, `db.sqlite3` contains:

1. **Standard Django tables (8 tables)**: `auth_group` (0), `auth_group_permissions` (0), `auth_permission` (499), `django_admin_log` (44), `django_content_type` (119), `django_migrations` (206), `django_session` (1,152), `sqlite_sequence` (88).
2. **10 Orphan / Legacy Tables** present in `db.sqlite3` that have **no active model in any `models.py`**:
    - `allocator_allocationrun` (2 rows) & `allocator_assignment` (4 rows, FK `allocation_id` -> `allocator_allocationrun.id`) — early prototype allocator tables.
    - `user_master` (0 rows) and 7 legacy profile tables (`admin_profile`, `doctor_profile`, `nurse_profile`, `patient_profile`, `receptionist_profile`, `staff_profile`, `therapist_profile` — all 0 rows, each with FK `user_id` -> `user_master.user_id`) — remnants of an older custom auth schema before `authentication.CustomUser` was adopted.

4. Cross-Database Logical Links (Soft Links Without SQL FKs)

Because SQLite (`default`) and SQL Server (`hospital_db`, `therapy_db`, `iptherapy_db`) are separate database engines, SQLite cannot enforce `FOREIGN KEY` constraints across them. Instead, the system joins records in Python using these logical keys:

Local Field (db.sqlite3)	Links To (External SQL Server / Other Table)	Purpose
Sheduling_patientsnapshot.pat_reg_no	Registration.pat_reg_no (therapy_db), OPMaster.NPat_Reg_No (hospital_db), Treatment.Pat_Reg_No (iptherapy_db)	Matches local scheduling snapshot to live hospital IP/OP patients and discharge status (pat_status='Y').
Patient_roombooking.patient_id , Patient_therapistbooking.patient_id , Patient_bookingcancellations.patient_id , MySql_Data_pull_visit.patient_id	Registration.pat_reg_no / OPMaster.NPat_Reg_No	Links bookings and cancellations to patient registration numbers.
Sheduling_therapyallocation.therapy_code	masters_therapymaster.therapy_code	Identifies the therapy definition.
Sheduling_bookingitem.item_code , store_TherapyItem.item_code , store_PrescriptionItem.item_code	masters_itemmaster.item_code or external ItemMaster.item_code (therapy_db)	Identifies medicines/oils/consumables.
masters_emergencyleave.employee_code	masters_therapistmaster.employee_code & authentication_therapistprofile.employee_code	Maps HRMS webhook/API emergency leaves to therapists (also cached via therapist_id FK).
Sheduling_doctorpatientpreferenceallowance.doctor_code , Sheduling_therapistpreferencerequest.treating_doctor_code	doctor_master.Doct_Code (therapy_db)	Maps local doctor users to hospital doctor codes.
Sheduling_patientsnapshot.pat_room_no	masters_stayingroom.number -> room_no	Maps staying room identifier to display room number.

5. Key Audit Findings, Anomalies & Technical Debt

When explaining this database in an audit, these are the most important architectural observations to highlight:

1. **Misleading `visit` Foreign Key Names on Booking Tables**:

- In , `visit` is a ForeignKey to **`Sheduling.Booking`** (not `MySql_Data_pull.Visit`).
- In , `visit` is a ForeignKey to **`Sheduling.TherapyAllocation`** (and `therapy` points to `masters.TherapyMaster`, whereas in `RoomBooking`, `therapy` points to `Sheduling.TherapyAllocation`).
- *Note*: The `RoomBooking.cancel()` and `TherapistBooking.cancel()` methods even try to pass `visit=self.visit` into `store_Therapy.objects.filter(visit=self.visit)` — which expects a `MySql_Data_pull.Visit`, highlighting legacy refactoring drift.

2. **Split Therapist Master References (`Patient.Therapist` vs `masters.TherapistMaster`)**:
   - `Sheduling.BookingAssignment.therapist` (20,105 rows) and `Patient.TherapistLeave.therapist` (105 rows) still have Foreign Keys pointing to legacy **`Patient.Therapist`** (`therapist_code` PK, 120 rows).
   - Meanwhile, `Patient.TherapistBooking` (21,138 rows), `Sheduling.TherapistContinuityPlan` (19,174 rows), `Sheduling.PatientPreferredTherapistRoster`, `Sheduling.HelpingTherapistAllocationLog`, and `masters.TherapistLeaveMaster` point to **`masters.TherapistMaster`** (`id` PK, 124 rows).
3. **Split TimeSlot Master References (`Patient.TimeSlot` vs `masters.SlotMaster`)**:
   - `MySql_Data_pull.store_Therapy.preferred_time_slot` and `Patient.AllocationAttempt.requested_time_slot` still reference legacy **`Patient.TimeSlot`** (7 rows).
   - All active scheduling (`Sheduling.TherapyAllocation`, `Booking`, `BookingAssignment`, `RoomBooking`, `TherapistBooking`, `TherapistContinuityPlan`, `ShiftSlotMapping`) uses **`masters.SlotMaster`** (9 rows).
4. **Multiple Overlapping Audit & Change-Tracking Tables**:
   - `authentication_auditlog` (21,350 rows) and `Sheduling_auditlog` (17,631 rows) are two separate `AuditLog` models serving system/auth and scheduling actions respectively.
   - Additional specialized audit/history tables exist: `Sheduling_schedulingauditlog` (11 rows), `Sheduling_bookingreschedulehistory` (11 rows), `Sheduling_allocationchange` (1 row), `Sheduling_prescriptionchangelog` (19,363 rows), `Patient_bookingcancellations` (76 rows), `masters_therapistoncalllog` (6 rows), and `Sheduling_helpingtherapistallocationlog` (1 row).
5. **Denormalized Snapshot & Dual-Storage Patterns (Intentional for Historical Integrity)**:
   - Tables like `TherapyAllocation`, `PrescriptionAllocation`, `Booking`, `BookingAssignment`, `PatientPreferredTherapistRoster`, and `SchedulingAuditLog` intentionally store JSON snapshots (`items_groups`, `required_therapist_roles`, `therapist_codes`,

`before_state` / `after_state`) alongside FKs/M2Ms so historical schedules remain intact even if masters or external hospital records change.

- `BookingAssignment` denormalizes `booking_date` and `booking_timeslot` from `Booking` specifically to enforce a database-level `UniqueConstraint` (`ux_assignment_therapist_date_timeslot`) preventing double-booking of a therapist in the same date + slot.
- Similarly, `RoomBooking` and `TherapistBooking` use `UniqueConstraint(..., 'status')` so cancelled rows (`CANCELED`) release the slot for new `BOOKED` rows without deleting history.

6. **Empty / Unused & Orphan Tables**:
   - **10 orphaned SQLite tables** (`allocator_*`, `user_master`, `*_profile`) exist in `db.sqlite3` with no corresponding Django models.
   - Several Django models have `0` rows in `db.sqlite3` (e.g., `Sheduling_prescriptionitem`, `Sheduling_cancellationrequest`, `Sheduling_therapynotification`, `Sheduling_prescriptionnotification`, `Sheduling_therapistpreferencerequest`, `masters_therapistdeputation`, `Patient_roomname`), indicating either unused/disabled features (such as WebSocket notifications) or features awaiting usage.
7. **SQLite Concurrency Consideration**:
   - Running all operational writes (262k-row `PatientHistory`, 20k+ `Booking` / `AuditLog` tables, plus Celery background tasks) on a single `db.sqlite3` file relies on a 30-second busy timeout in `settings.py` (`'timeout': 30`). For higher concurrency or multi-worker deployments, migrating `default` from SQLite to PostgreSQL/SQL Server is a standard audit recommendation.